GESTIÓN DEL PROCESO DE DESARROLLO DE SOFTWARE

Práctica 2

Mini-proyecto: automatización de TiendaUT

La segunda práctica se aplica a TiendaUT, el proyecto disponible en el ZIP, siguiendo la indicación de trabajar con ese archivo. Es una adaptación del mini-proyecto: si el profesor exige el sistema propio del equipo, deberá aplicarse a ese sistema.

Objetivo: cubrir recorridos críticos, casos negativos, API, análisis de calidad y configuración de CI. Requisitos: capítulo 19, página 53 de la guía. Fecha: 15 de octubre de 2026.

Requisito	Implementación y evidencia
Ocho E2E y dos recorridos completos	Matriz de casos de la página siguiente; FC-01 y FC-02.
Dos negativos y una API	Contraseña incorrecta, campos vacíos, API 500 y contrato de productos.
Page Object y preparación común	LoginPage y CatalogoPage; beforeEach y fixture loginPage.
Consola, red y axe	Tres auditorías originales y adjunto calidad.json.
baseURL, webServer y trace	playwright.config.ts; trace retain-on-failure.
Pipeline y reporte	GitHub Actions en verde y reporte publicado como artefacto.
Bitácora completa	BITACORA.md, INDICE-EVIDENCIAS.md y trazas históricas.

Los dos recorridos completos fueron añadidos expresamente. Se verificó toda la suite ampliada: 345/345 en tres navegadores. Las ocho pruebas documentales aparte capturan los estados después de verificarlos; no aumentan artificialmente el recuento de aceptación.

1. Diseño y trazabilidad de casos

ID / tipo	Pasos y resultado esperado	Archivo
E2E-01 / crítico	Inicio > login válido > catálogo con cuatro productos y contador 0.	flujos-completos.spec.ts / FC-01
E2E-02 / crítico	Inicio > catálogo > buscar monitor y agregar > buscar mouse y agregar > contador 2 > vaciar y verificar 0.	flujos-completos.spec.ts / FC-02
E2E-03 / negativo	Contraseña equivocada: alerta y permanencia en login.	login.spec.ts
E2E-04 / negativo	Campos vacíos: alerta y permanencia en login.	practica.spec.ts
E2E-05 / funcional	Filtrar monitor y comprobar exactamente un resultado.	carrito.spec.ts
E2E-06 / negativo	API 500: presentar aviso al usuario.	red-y-visual.spec.ts
E2E-07 / negativo	Red interrumpida: aviso y cero excepciones JavaScript.	regresiones.spec.ts
E2E-08 / funcional	Inicio: título TiendaUT y encabezado correcto.	inicio.spec.ts
API-01 / API	GET exitoso: cada producto tiene nombre y precio válido mayor que 0.	practica.spec.ts

Aislamiento y criterios

Cada prueba recibe un contexto nuevo. Los recorridos completos preparan el inicio con beforeEach; el fixture loginPage y los Page Objects encapsulan la pantalla. Se usan getByRole, getByLabel y testid acordados; las aserciones reintentan, sin waitForTimeout. Los datos de acceso son únicamente los de la tienda didáctica.

TiendaUT no implementa pago ni confirmación de pedido. Por ello se verifica acceso y gestión del carrito, sin inventar un checkout inexistente.

2. FC-01: acceso y catálogo cargado

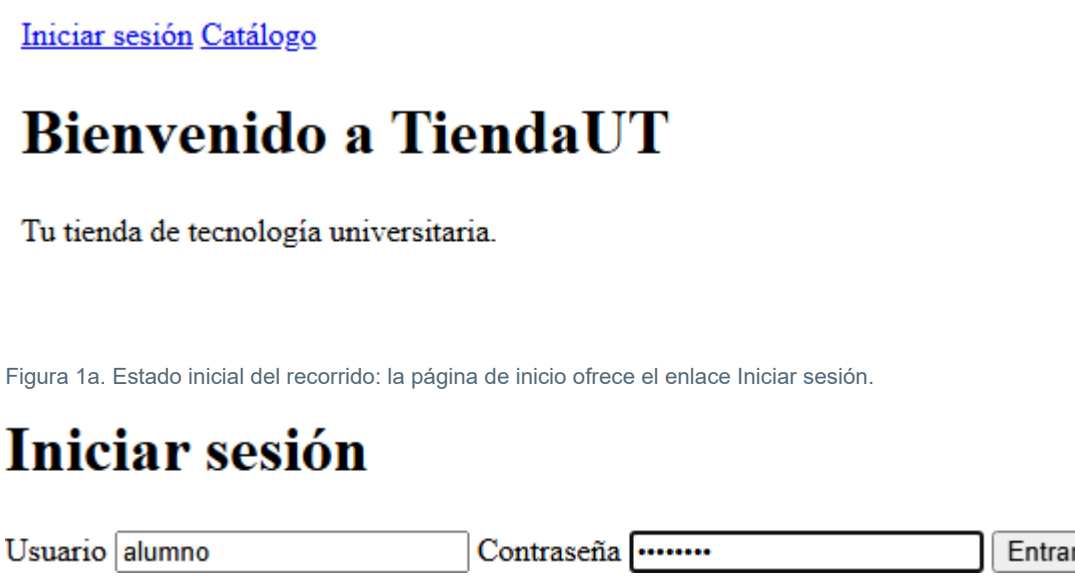


Figura 1a. Estado inicial del recorrido: la página de inicio ofrece el enlace Iniciar sesión.

Figura 1. El recorrido comienza en Inicio, abre Iniciar sesión y completa alumno / contraseña de demostración.



Figura 2. Después de Entrar se verifica catalogo.html, encabezado Catálogo, cuatro productos y contador inicial 0.

Resultado esperado y obtenido: navegación válida y catálogo cargado. Implementación: LoginPage y CatalogoPage, test FC-01. El resultado se verifica en Chromium, Firefox y WebKit durante cinco repeticiones por navegador.

3. FC-02: búsqueda y gestión del carrito



Figura 3. Tras buscar monitor, agregarlo, buscar mouse y agregarlo, el contador muestra 2.

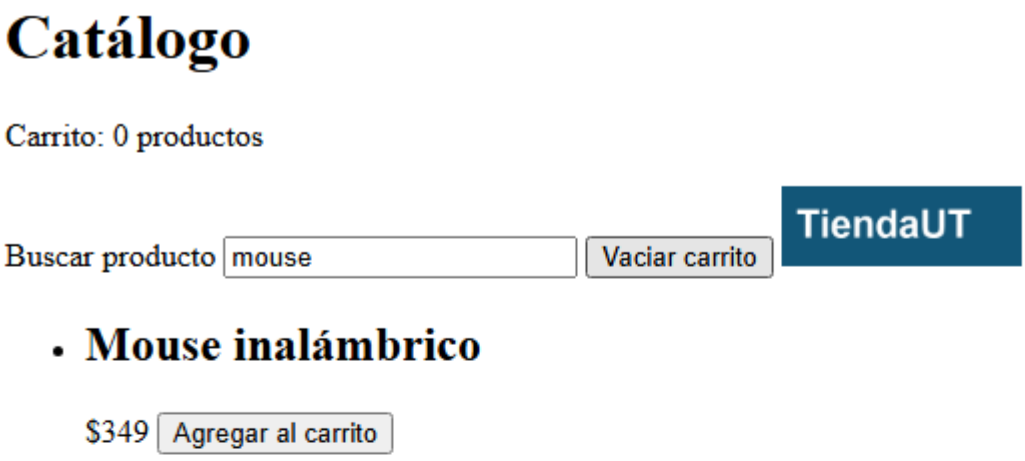


Figura 4. Se pulsa Vaciar carrito y se verifica contador 0. El flujo de aceptación también exige ausencia de pageerror.

Inicio > Catálogo > monitor > agregar > mouse > agregar > contador 2 > vaciar > contador 0. CatalogoPage concentra buscador, productos, contador y acciones. El flujo prepara y verifica su propio estado; no depende de otra prueba.

4. Casos negativos con resultado visible

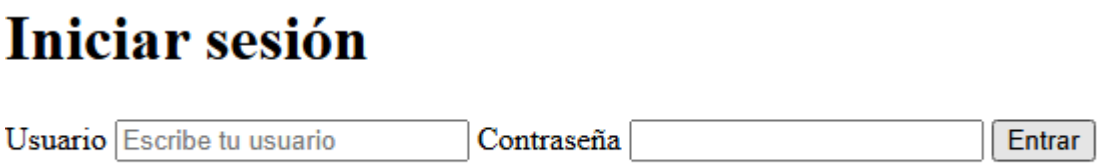


Figura 5. Formulario vacío: el sitio presenta la alerta de error y permanece en login. El caso con contraseña equivocada se verifica igualmente.



Figura 6. Respuesta 500 controlada: aparece una alerta de carga fallida. También existe una prueba que aborta la petición de red.

Las capturas muestran estados del navegador. La comprobación de URL, texto y errores se hace con aserciones dentro de Playwright y queda registrada en los reportes, no solamente a partir de la apariencia de la pantalla.

5. API y análisis de calidad

Pretty-print ☑


```
[
  {
    "id": 1,
    "nombre": "Teclado mecánico",
    "precio": 899
  },
  {
    "id": 2,
    "nombre": "Mouse inalámbrico",
    "precio": 349
  },
  {
    "id": 3,
    "nombre": "Monitor 24 pulgadas",
    "precio": 2799
  },
  {
    "id": 4,
    "nombre": "Audífonos USB",
    "precio": 459
  }
]
```


Figura 7. JSON real de la API. Cada producto se valida por nombre y precio; la captura no sustituye las comprobaciones del contrato.

Resultado de la auditoría automatizada


```
{
  "consola": [],
  "red": [],
  "violaciones": [],
  "reglasComprobadas": 24
}
```


El JSON anterior proviene del adjunto real de la prueba documental. Axe analiza el catálogo con los productos cargados. Cero violaciones detectadas no equivale a certificación completa de accesibilidad ni a una revisión de todas las pantallas.

Trazabilidad de defectos

Se mantiene la bitácora de todos los fallos del desarrollo: E1-E8, consola/red, accesibilidad, carrera del buscador, vaciado, error de API, referencia visual inicial y problemas de entorno. Cada entrada explica evidencia, clasificación, causa, corrección y verificación; la documentación principal se divide únicamente en estas dos prácticas.

6. Ejecución, evidencia y pipeline

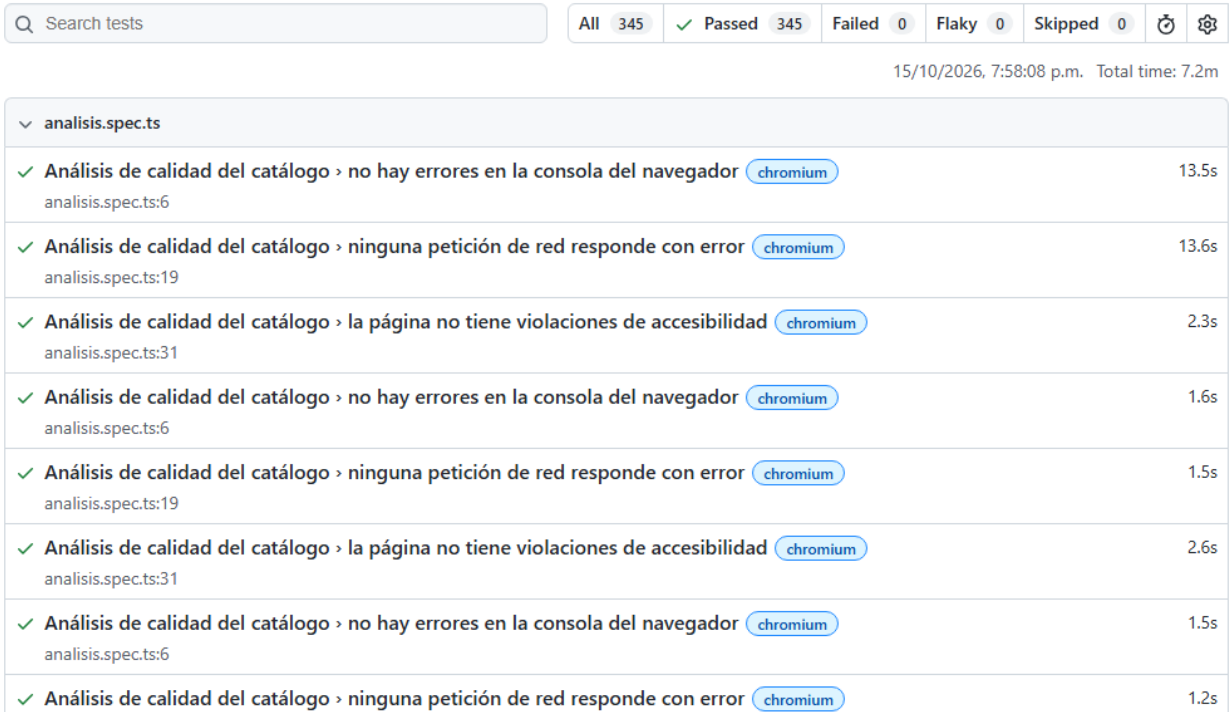


Figura 8. Resultado real común a las dos prácticas: 23 casos x 5 repeticiones x 3 navegadores = 345 aprobados.

Chromium 115/115; Firefox 115/115; WebKit 115/115. Cero fallos, flaky o saltos. ESLint sin errores. Resultado fuente: evidencias/estabilidad-ampliada/resumen.json. Evidencia documental: ocho pruebas separadas, con trazas y PNG.

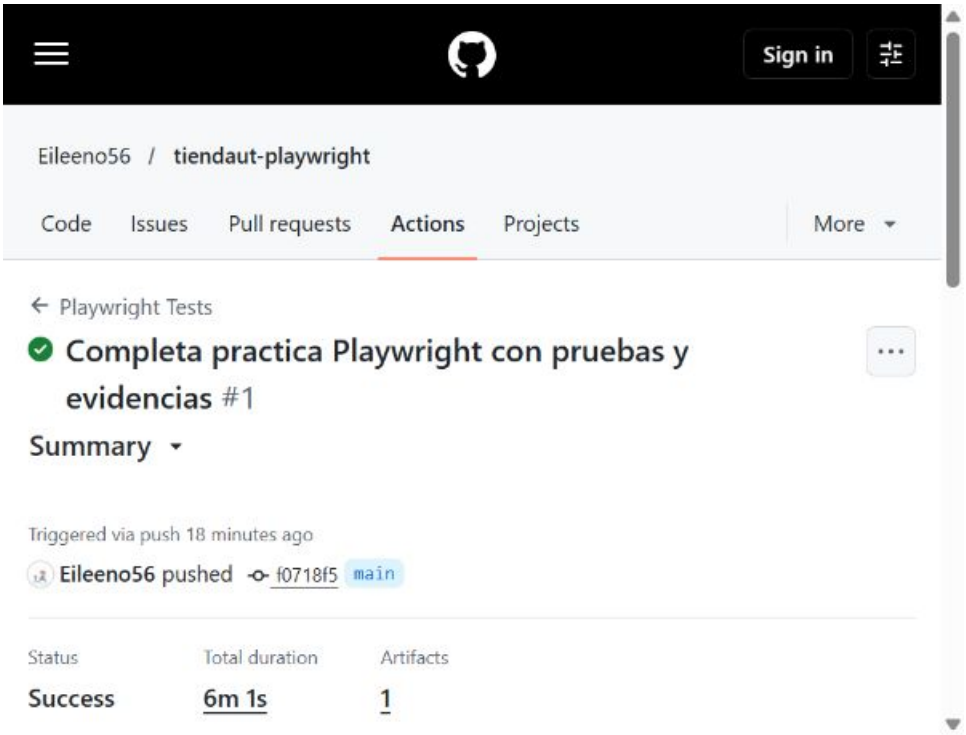
Reproducir y consultar

```
npm ci
npx playwright install
npm run lint
npm run test:estabilidad
npx playwright test --config=playwright.evidencias.config.ts
```

Integración continua comprobada

El repositorio ya está publicado y GitHub Actions terminó en verde con npm ci, lint y 345 pruebas. Se publicó y descargó el artefacto playwright-report. La captura de la página siguiente muestra GitHub Actions con estado Success. El repositorio es público y main exige el check test para integrar cambios.

GitHub: ejecución y artefacto verificados



Captura real de la página pública de GitHub Actions: estado Success y un artefacto publicado. El reporte de Playwright descargado y la segunda ejecución estricta también están incluidos en las evidencias.

Ejecución del 21 de octubre de 2026

Comprobación remota	Resultado
Instalación y ESLint	Correctos
Chromium / Firefox / WebKit	115 / 115 en cada navegador
Total / fallos / flaky / saltos	345 / 0 / 0 / 0
Artefacto playwright-report	Publicado y descargado; JSON y logs comprobados

Repositorio: https://github.com/Eileeno56/tiendaut-playwright

Ejecución: https://github.com/Eileeno56/tiendaut-playwright/actions/runs/39695106242

La verificación y los adjuntos están en evidencias/github/. El repositorio es público y el docente puede abrirlo sin invitación. Main exige el check test. La segunda ejecución también pasó 345/345 sin actualizar referencias: actions/runs/39696522338. El bloqueo inicial del plan privado queda registrado como antecedente, no como estado actual.